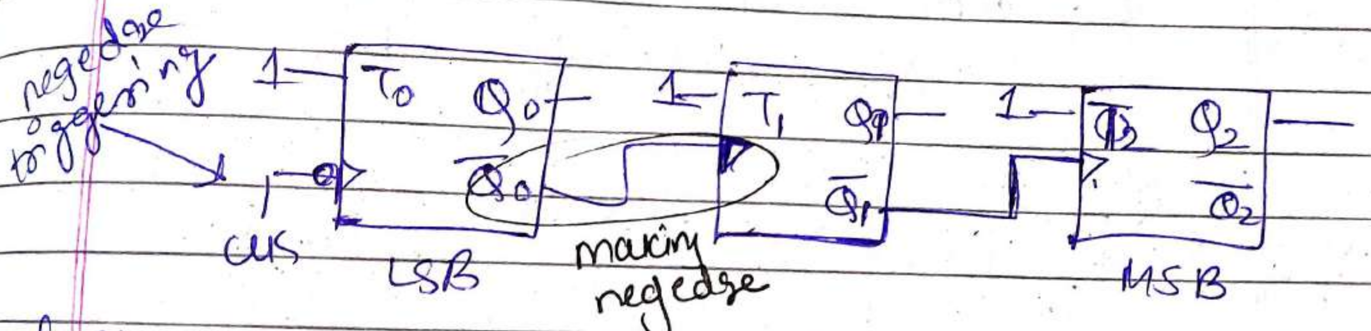
Prerna Agarwal

Counters

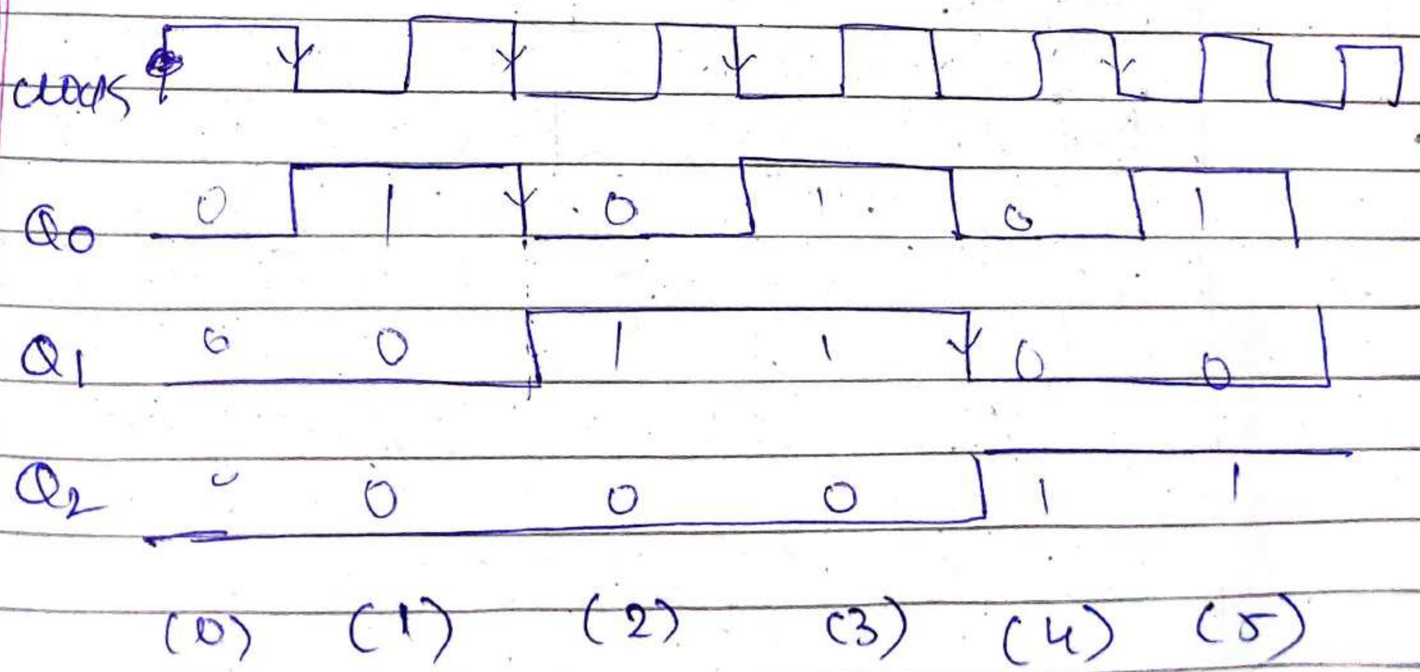
1Q) Design 3bit Asynchronous UP counter using T flipflops.

→ Binary Asynchronous counter
count 0 to 7

3 Bit means 3 flipflops needed
Total states : $2^3 = 8$



waveform

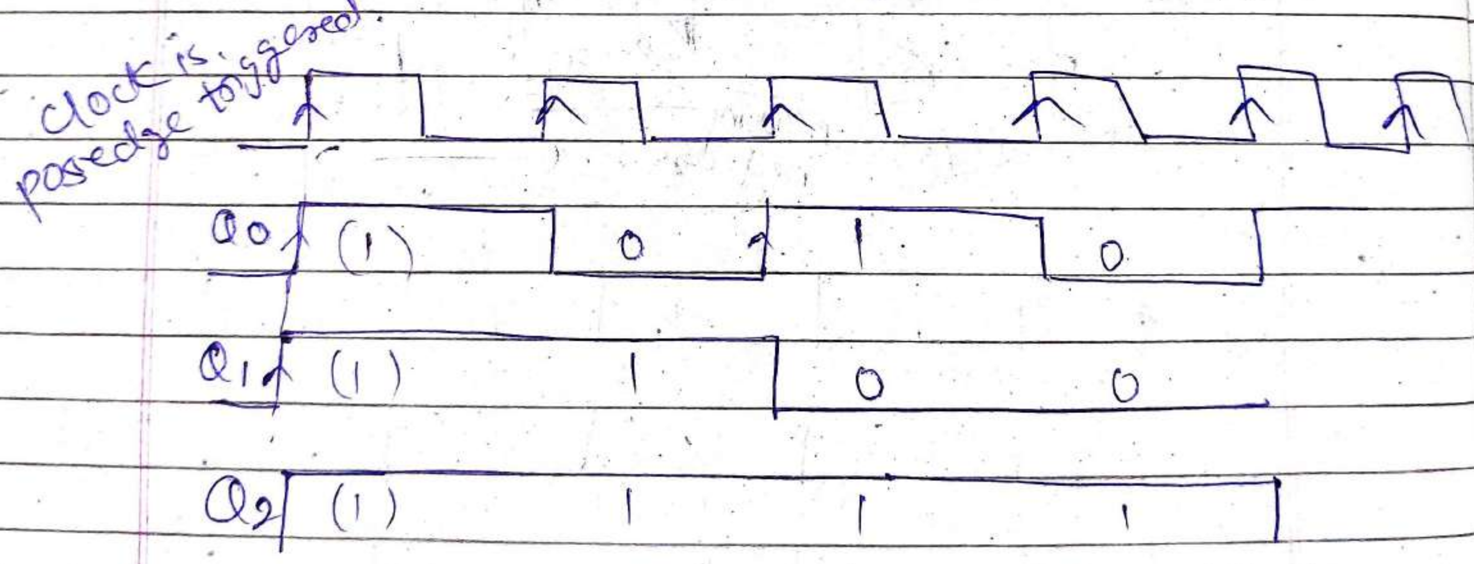


2 Q ~~Sta~~ Why called as Ripple ??

Becoz the clock pulse is applied to the first flip-flop. and output of each flipflop act as clock for the next one, so the state changes propagate sequentially from one stage to next.

Therefore called as ripple effect.

3 Q if clock connection is ~~…~~ connected Q signal of 3 bit Async. Counter



(7) (6) (5) (4) . . .

down counter.

4 Q Design 3 bit up-down counter with a mode control input

M=0 (up counting) → clk connected to Q

M=1 (down counting) → clk connected to $\overline{Q}$

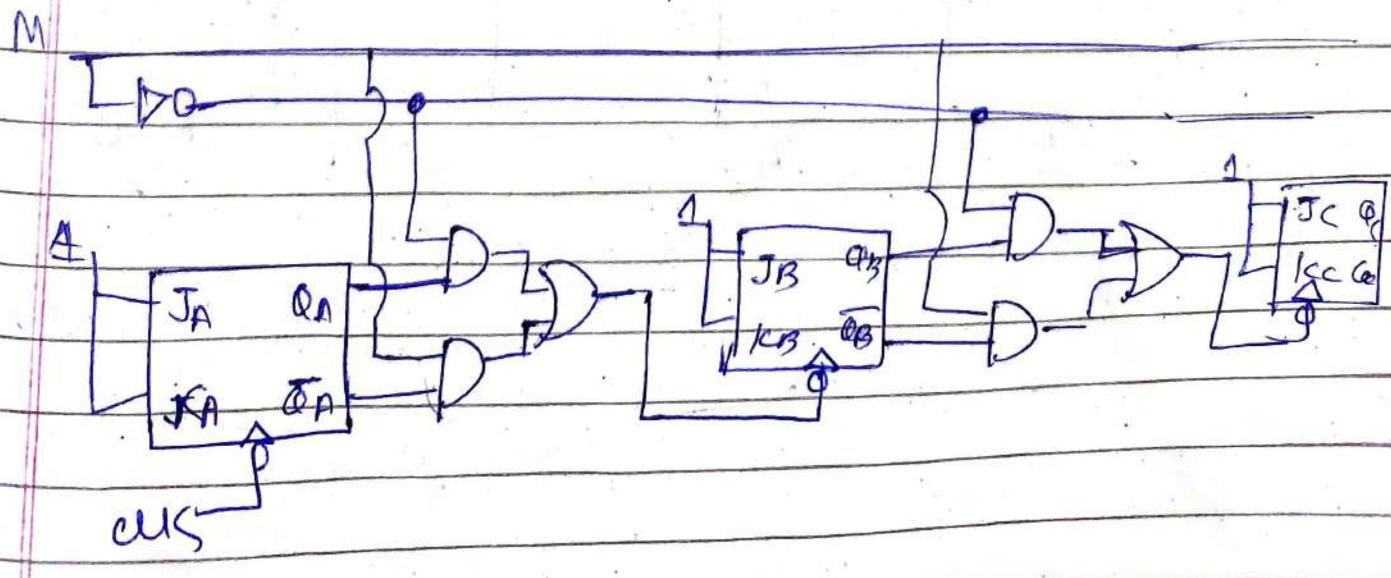
→ first obtain combinational logic for up/down functionality

	M	Q	$\overline{Q}$	Y
up	0	0	0	0
	0	0	1	0
	0	1	0	1
	0	1	1	1
down	1	0	0	0
	1	0	1	1
	1	1	0	0
	1	1	1	1

M \ $Q\overline{Q}$	00	01	11	10
0			1	1
1		1	1	

$$Y = \overline{M}Q + M\overline{Q}$$

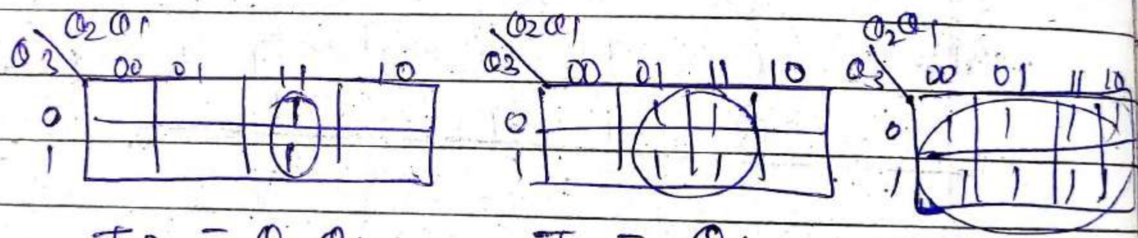
implement in between each flip-flops



5Q Design 3 bit posedge ~~triggered~~ synchronous up Counter

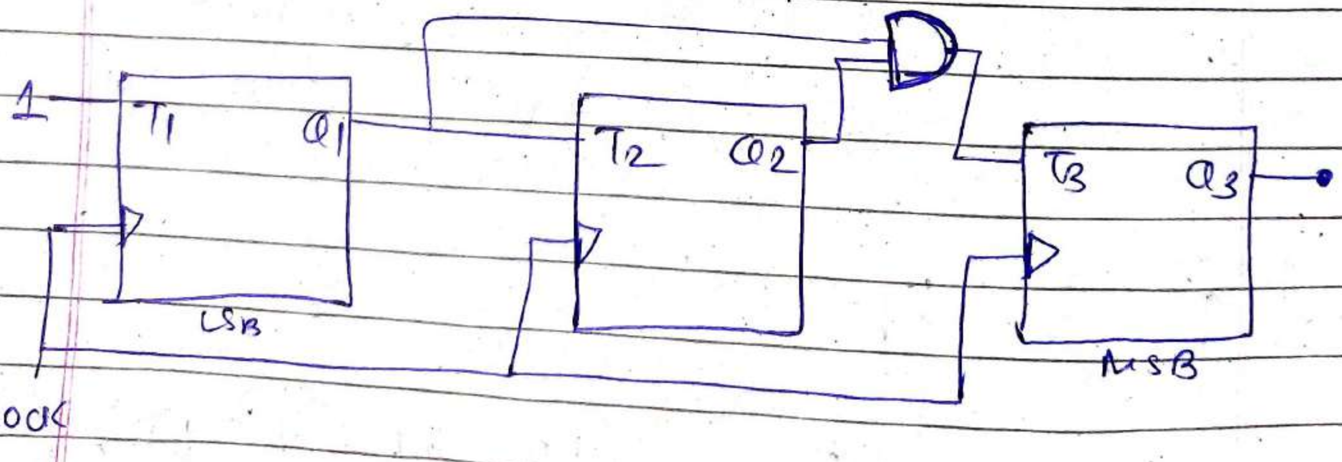
3 bit
∴ 3 flipflops
$2^3 = 8$

P.S (MSB … LSB)			N.S			Flip flop input		
Q_3	Q_2	Q_1	Q_3^+	Q_2^+	Q_1^+	T_3	T_2	T_1
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



$T_3 = Q_2Q_1$ $T_2 = Q_1$

$T_1 = 1$



6Q Why do we need input equations while designing Synchronous Counter?

→ if all T flip flops
$T_1 = T_2 = T_3 = 1$ (set)

then every clock edge will change their state all flipflops as they all receives same clock edge simultaneously.

	$Q_3 Q_2 Q_1$
↑	000
↑	111
↑	000

} producing incorrect sequence.

Therefore, we desire input Equation of ~~at~~ each flipflops to achieve the desired counting sequence.

7Q Why flipflops are in toggle state while designing Asynchronous Counter

→ The purpose is to produce a binary count sequence

$Q_3 Q_2 Q_1$
000
001
010
011
100
101
110
111

$Q_3 Q_2 Q_1$

Q_1 toggles every clock cycles

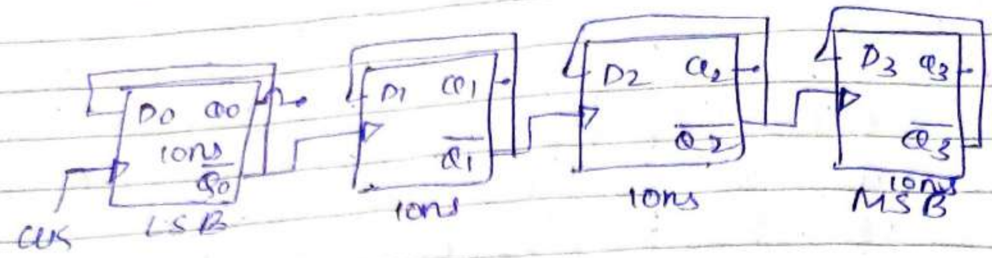
Q_2 toggles every time Q_1 goes $1 \to 0$

Q_3 toggles every time Q_2 goes $1 \to 0$

This makes up count sequence

8 Q Why does an Asynchronous Counter produce Glitches ?

→



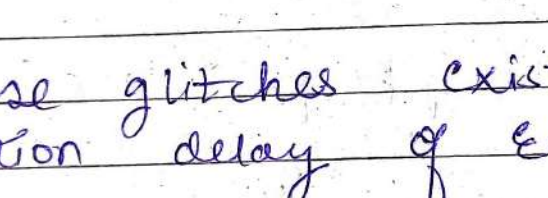
suppose current state

	Q3	Q2	Q1	Q0		
	0	1	1	1	(7)	
				toggle		
after 10ns	0	1	1	0	→ (6)	intermediate states are wrong ∴ glitches
after 20ns	0	1	0	0	→ (4)	
30ns	0	0	0	0	— (0)	
40ns	1	0	0	0	(8)	

These glitches exist due to propagation delay of Each flipflop

Each flipflop changes state at different time.

Combinational ckt

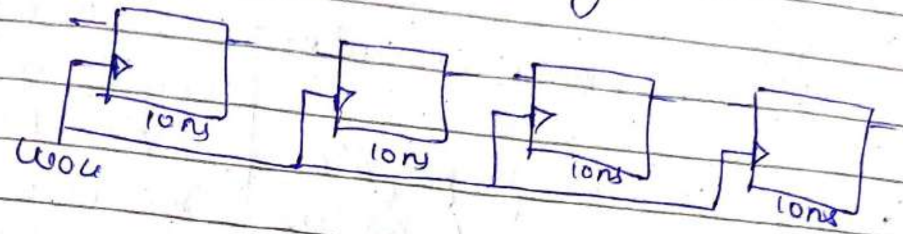


decoder expects (8) after (7)
But it briefly sees 0110, 0100, 0000
i.e. activate wrong output 6, 4, 0 line instead of (8)
∴ False decoding

Counter

9 Q How does a synchronous counter solve this Glitch problem?

→ All flipflops receive the same clock simultaneously.



They all have propagation delay so after 10ns they all change their state.

current state → $Q_3 Q_2 Q_1 Q_0$ 0 1 1 1 (7)

clock edges ↑ — unchanged —

after 10ns

1 0 0 0 (8) all flipflops change their state at same time

Only 1 Propagation delay is seen

→ Although each flipflops still has (tpd) propagation delay, they all change their state at same time and settle to next state together after 1 propagation delay.

So, This helps in Eliminating ripple induced glitches

10 Q If each flipflop has a propagation delay of 10ns.
Max delay ?? for a 4bit Ripple counter & 4 bit Synchronous counter?

→ Asynchronous countr

clock must propagate through each flipflop

Once All receive clock, tpd of each ~~prog~~ accumulate

FF0 output change after 10ns

FF(1) " " 20ns

FF 2 " " 30ns

FF3 " " 40ns

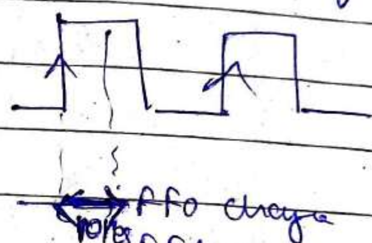
Max dely = 40ns

= n×10ns

n = 4

Synchronous.

All flipflops receive same clock simultaneously



10ns FF0 change

FF1 change

FF2 change

FF3 change

Max delay = 10ns

Q if clock = 10MHZ, output frequency of a 4 bit Synchronous counter?

→ 4 bit means 4 flipflops needed

Total states → 0 to $2^4 - 1$

= 0 to 15

Page No. | Date

clarify output frequency. → frequency at Particular flip flop output?

→ frequency at MSB bit? (final stage)

clock	Q_3 (MSB)	Q_2	Q_1	Q_0 (LSB)
↑	0	0	0	0
↑	0	0	0	1
↑	0	0	1	0
↑	0	0	1	1
↑	0	1	0	0
↑	0	1	0	1
↑	0	1	1	0
↑	0	1	1	1
↑	1	0	0	0
↑	1	0	0	1
↑	1	0	1	0
↑	1	0	1	1
↑	1	1	0	0
↑	1	1	0	1
↑	1	1	1	0
↑	1	1	1	1

$f_{Q_0} = ?$

$T_{Q_0} = 2T_{clk}$

$f_{Q_0} = \frac{f_{clk}}{2}$

$T_{Q_1} = 4T_{clk}$

$f_{Q_1} = \frac{f_{clk}}{4}$

$f_{Q_2} = \frac{f_{clk}}{8}$

$f_{Q_3} = \frac{f_{clk}}{16}$

for n bit Counter (Binary)

$$\boxed{f_{Q_0} = \frac{f_{clk}}{2^{n+1}}}$$

12 Q if clock frequency = 10MHZ
output frequency of MSB bit for a MOD 5 down counter?

→ $5 \ll 2^n$

$5 \ll 2^3$ 3 bit needed
∴ 3 flip flops

But this Non Binary Asynchronous counter, count Total 5 states

CLK	Q_2	Q_1	Q_0
	1	1	1
	1	1	0
	1	0	1
	1	0	0
	0	1	1

$f_{Q_0} = \frac{f_{clk}}{2}$

$f_{Q_1} = ?$

$T_{Q_1} = 5 T_{clk}$

$f_Q = \frac{f_{clk}}{5}$

f_{Q_2} ?

$T_{Q_2} = 5 T_{clk}$

$f_{Q_2} = \frac{f_{clk}}{5}$

for a MOD N counter

$$\boxed{f_Q = \frac{f_{clk}}{N}}$$

Date

13 Q for a 4 bit Synchronous counter What is MOD ?

→ Modulus of counter : Total no. of unique states the counter goes

4 bit
$\therefore 2^4 = 16$
MOD 16

To make it MOD 10

→ MOD 10 means count from 0 to 9

10 } should not count
11
12
13 so obtain reset signal
14 reset from 10 decimal Num
15

$(10)_{10} \rightarrow (1010)_2$

$\boxed{reset = Q_3 \overline{Q_2} Q_1 \overline{Q_0}}$

14 Q How many flipflops are needed for a MOD 32 up counter?

→ MOD 32 means counter ~~starts~~ counts from 0 to 31

$32 \ll 2^n$

$\leq 2^5$ $\boxed{\therefore \text{ 5 flip flops}}$

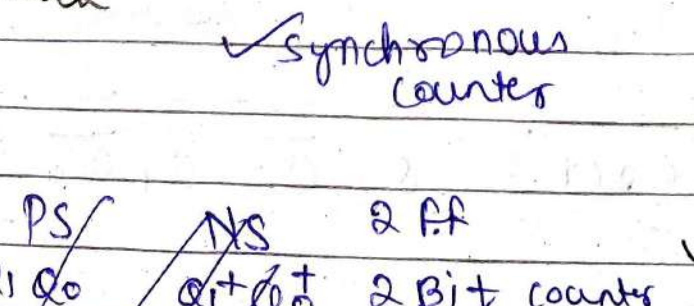
b) for MOD 20

$20 \ll 2^n$

$n = 5 \longrightarrow$ 5 flipflops

$2^5 = 32$ states But counter counts upto 19.

15 Q



MOD N?

✓ synchronous counter

JK flipflop characteristic

$Q^+ = J\overline{Q} + \overline{K}Q$

PS		NS	
Q_1	Q_0	Q_1^+	Q_0^+
0	0		
0	1		
1	0		
1	1		

2 ff

2 Bit counter

Total states $= 2^2 = 4$

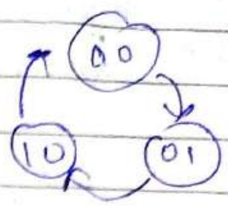
✓ $Q_1^+ = J_1\overline{Q_1} + \overline{K_1}Q_1$

$= Q_0\overline{Q_1} + 0$

✓ $Q_0^+ = \overline{Q_1}\,\overline{Q_0} + 0$

$Q_1 Q_0$ PS	$Q_1^+ Q_0^+$ NS	
0 0	0 1	3 used states
0 1	1 0	
1 0	0 0	
1 1	0 0	

4 states



$\therefore$ MOD 3

16 Q A 4 bit up-down counter is at state 1001

if mode = UP next 3 states?
mode = DOWN next 3 states?

→ 4 bit → 4 flip flops

Total state $= 2^4 = 16$
count from 0 to 15

Mode = UP

PS → 1001 $= (9)_{10}$
N.S. 1010 $= (10)_{10}$
1011 $= (11)_{10}$
1100 $= (12)_{10}$

→ if counter state 1111 ↓ wrap 0000

Mode = down

PS → 1001 $= (9)_{10}$
N.S 1000 $= (8)_{10}$
0111 $= (7)_{10}$
0110 $= (6)_{10}$

if counter is at 0000 ↓ rollover 1111

17 Q How do you modify upcounter To work as Down Counter?

- upcounter

$Q_1 Q_2 Q_3$
000
001
010
011
100
101
110
0111
1000

Higher order of toggle when lower Bits are 1 — toggle

down counter

$Q_3 Q_2 Q_1 Q_0$
1000 (8)
↓
0111 (7)

in down count Higher bit changes when all lower bits become 0

1) If Asynchronous counter

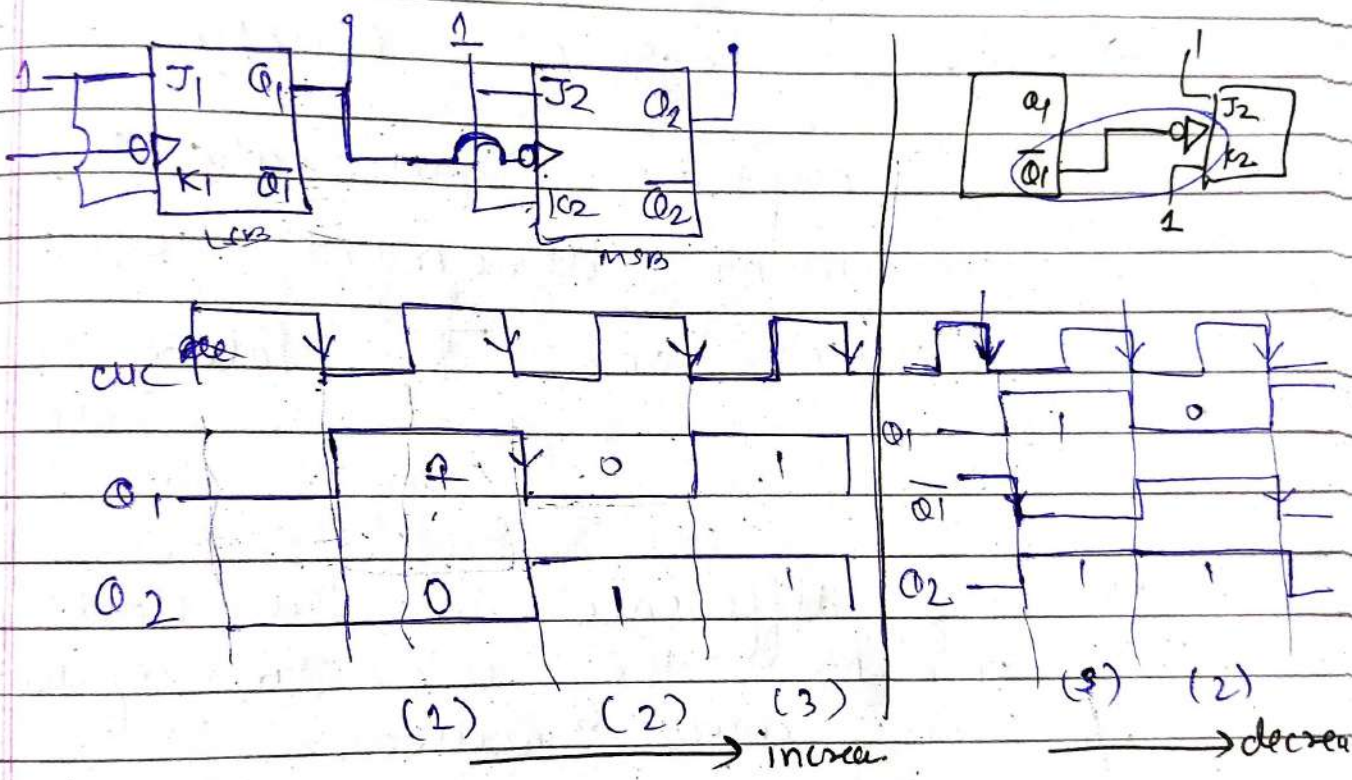
1) positive Edge triggering
up count → $\bar{Q}$ connected To clock for next stage
down count → Q → clk

2) Negedge Edge triggering
Q → clk gives upcount
$\bar{Q}$ → clk gives downcount

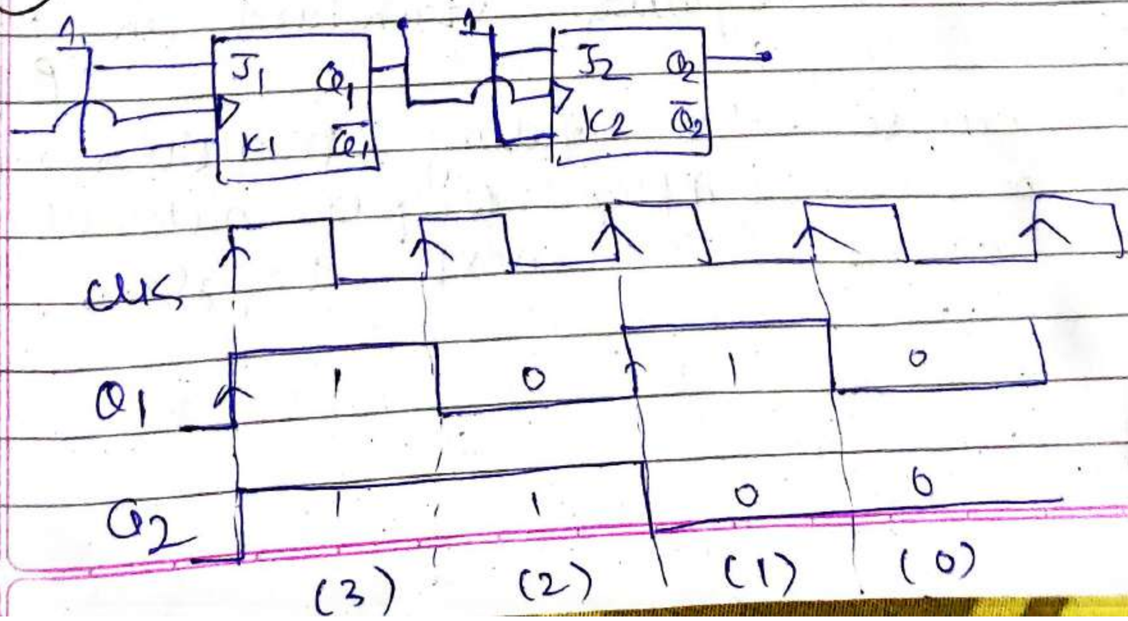
8 Q Compare posedge triggered counter & negedge triggered counter?

→ 2 Bit Ripple ~~up~~ counter

1. (-ve edge triggered) Flipflops



2. (+ve edge triggered Flipflop)



→ Posedge triggered counter

↳ captures input data & can increment / decrement state on rising edge of clock

→ Negedge triggered counter.

↳ captures input data & can increment / decrement count value (state) on falling edge of clock

→ only difference is the clock transition moment they use to update their count value.

19 Q) Why negedge triggered counter selected as a default standard in Ripple up counter?

→ Because the falling transition of previous flipflop's Output naturally drives the next toggle.

Q Design BCD (decade) up ~~count~~ Ripple Counter

$10 << 2^n$

$\therefore n = 4$ 4bit → 4 flipflops need

decade counter count from 0 to 9

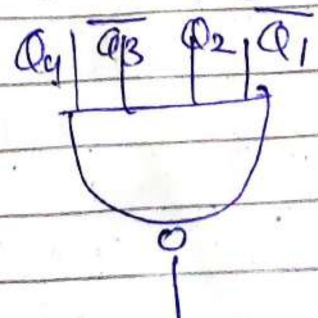
from 10, onwards → reset the decade counter.

∴ combinational logic required

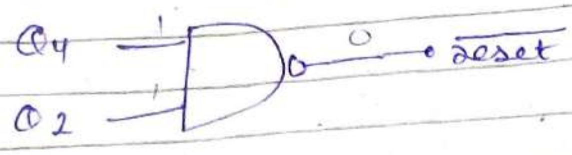
CLK	$Q_4 Q_3 Q_2 Q_1$ (MSB … LSB)	$\overline{Reset}$
*	0 0 0 0	1
↑	0 0 0 1	1
↑	0 0 1 0	1
↑	0 0 1 1	1
↑	0 1 0 0	1
↑	0 1 0 1	1
↑	0 1 1 0	1
↑	0 1 1 1	1
↑	1 0 0 0	1
↑	1 0 0 1	1
↑	1 0 1 0	0 ✓
↑	1 0 1 1	X
↑	1 1 0 0	X
↑	1 1 0 1	X
↑	1 1 1 0	X
↑	1 1 1 1	X

$Q_4 Q_3 Q_2 Q_1$

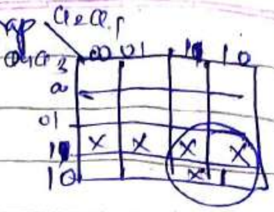
1 0 1 0

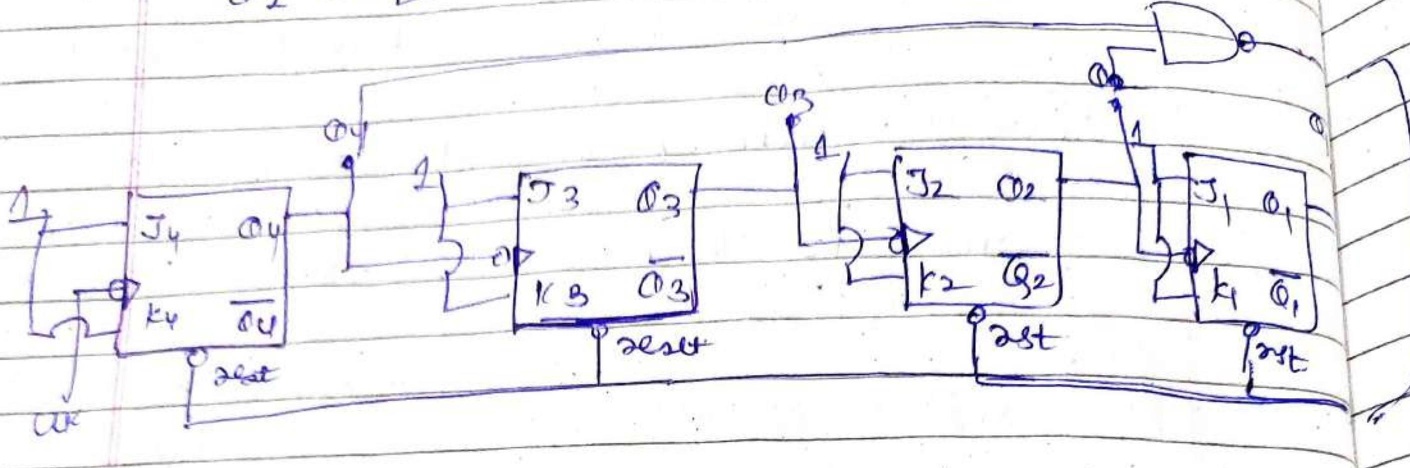


or directly



By Kmap





Q BCD synchronous up counter.

PS	NS	Flip Flop inputs
$Q_4 Q_3 Q_2 Q_1$	$Q_4^+ Q_3^+ Q_2^+ Q_1^+$	$T_4 T_3 T_2 T_1$
0 0 0 0	0 0 0 1	0 0 0 1
0 0 0 1	0 0 1 0	0 0 1 1
0 0 1 0	0 0 1 1	0 0 0 1
0 0 1 1	0 1 0 0	0 1 1 1
0 1 0 0	0 1 0 1	0 0 0 1
0 1 0 1	0 1 1 0	0 0 1 1
0 1 1 0	0 1 1 1	0 0 0 1
0 1 1 1	1 0 0 0	1 1 1 1
1 0 0 0	1 0 0 1	0 0 0 1
1 0 0 1	$\boxed{0 0 0 0}$	1 0 0 1
1 0 1 0	X X X X	X X X X
1 0 1 1	X X X X	X X X X
1 1 0 0	X X X X	X X X X
1 1 0 1	X X X X	X X X X

Q_4Q_3 \ Q_2Q_1	00	01	11	10
00				
01			1	
11	X	X	X	X
10		1	X	X

$T_4 = Q_4Q_1 + Q_3Q_2Q_1$

~~$T_4 = Q_4 Q_2 Q_1 + Q_3Q_2Q_1$~~

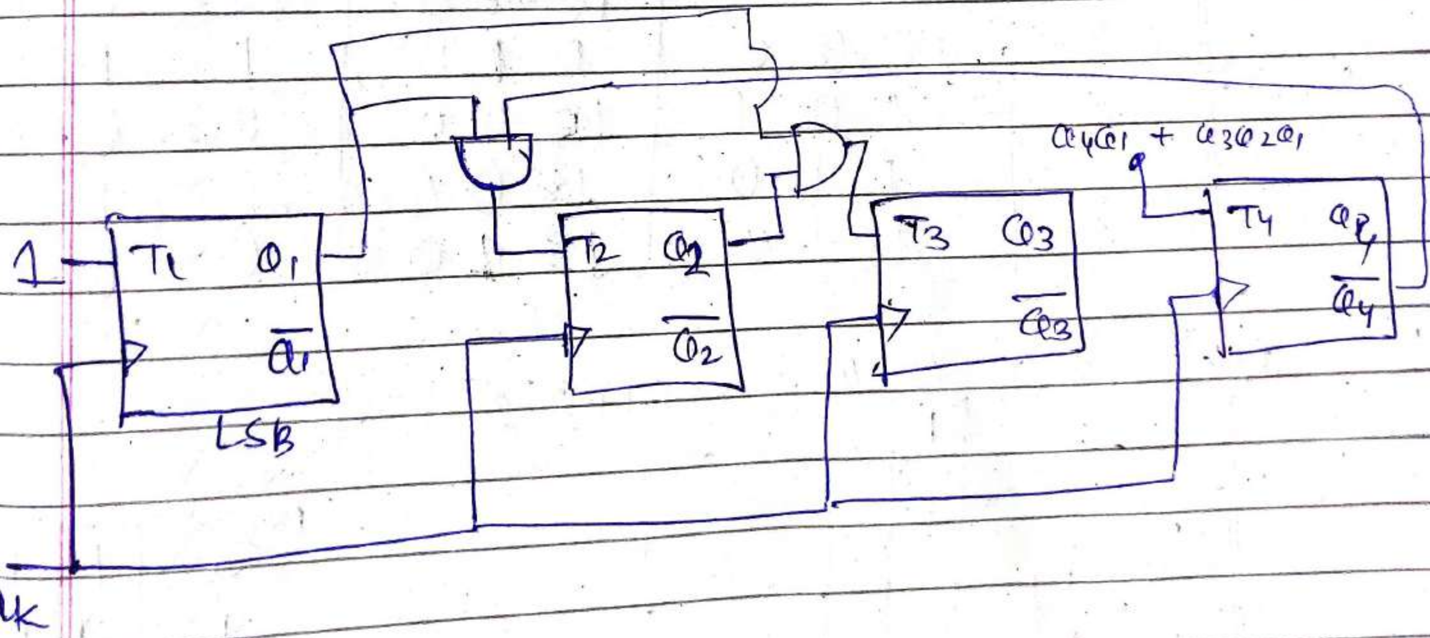
Q_4Q_3 \ Q_2Q_1	00	01	11	10
00			1	
01			1	
11	X	X	X	X
10			X	X

$T_3 = Q_2Q_1$

Q_4Q_3 \ Q_2Q_1	00	01	11	10
00		1	1	
01		1	1	
11	X	X	X	X
10			X	X

$T_1 = 1$

$T_2 = \overline{Q_4} \cdot Q_1$



Q 3 Bit up-down Synchronous Counter

M	PS Q_3	Q_2	Q_1	NS Q_3^+	Q_2^+	Q_1^+	Flip Flop input T_3	T_2	T_1
0	0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	0	0	1	1
0	0	1	0	0	1	1	0	0	1
0	0	1	1	1	0	0	1	1	1
0	1	0	0	1	0	1	0	0	1
0	1	0	1	1	1	0	0	1	1
0	1	1	0	1	1	1	0	0	1
0	1	1	1	0	0	0	1	1	1
1	0	0	0	1	1	1	1	1	1
1	0	0	1	0	0	0	0	0	1
1	0	1	0	0	0	1	0	1	1
1	0	1	1	0	1	0	0	0	1
1	1	0	0	0	1	1	1	1	1
1	1	0	1	1	0	0	0	0	1
1	1	1	0	1	0	1	0	1	1
1	1	1	1	1	1	0	0	0	1

up count (M = 0)

down count (M = 1)

$MQ_3 \backslash Q_2Q_1$	00	01	11	10
00			1	
01			1	
11	1			
10	1			

$T_3 = M\bar{Q_2}\bar{Q_1} + \bar{M}Q_2Q_1$

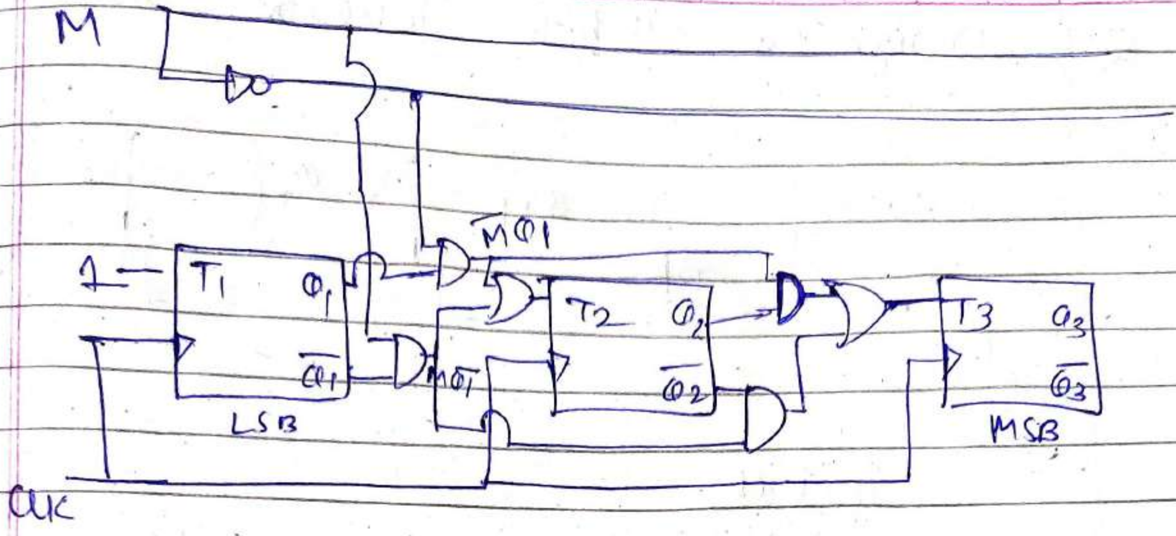
$T_1 = 1$

$T_2 =$

$MQ_3 \backslash Q_2Q_1$	00	01	11	10
00		1	1	
01		1	1	
11	1			1
10	1			1

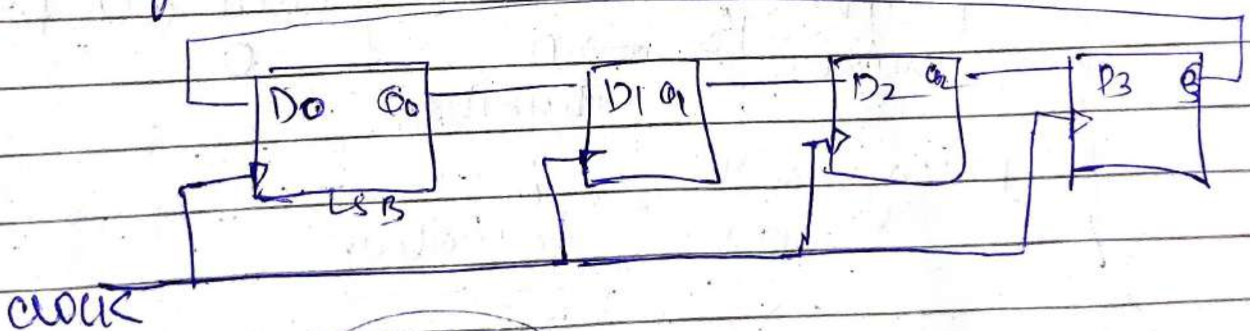
$= M\bar{Q_1} + \bar{M}Q_1$

Page No.
Date



Q Design a 4 bit Ring Counter using D flipflops. Initial state = 1000, next 4 states?

→ Ring Counter is a shift register in which output of last flip-flop is fed back to the input of 1st FF.



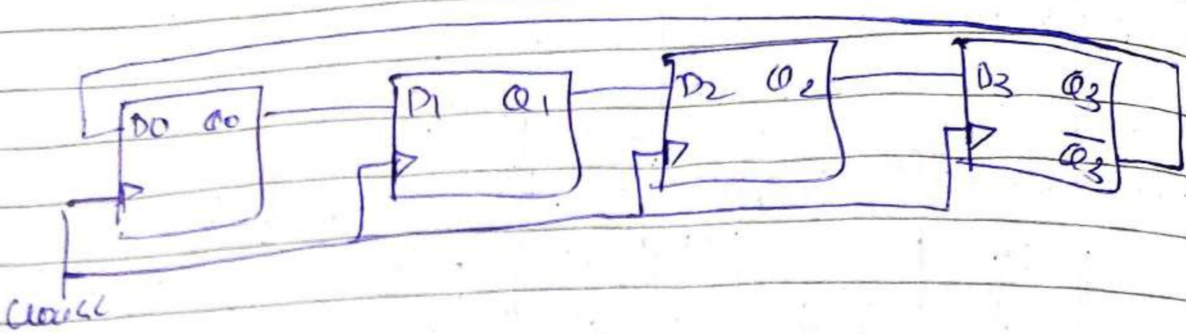
Q_3	Q_2	Q_1	Q_0
1	0	0	0
0	0	0	1
0	0	1	0
0	1	0	0
1	0	0	0

freq of output $= \frac{f_{clk}}{4}$

n bit ⟶ n sequence length

Q Design a 4 bit Johnson Counter

→



initial

Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	1
0	1	1	1
1	1	1	1
1	1	1	0
1	1	0	0
1	0	0	0
0	0	0	0

✓ 8 unique states

for n bit (n flipflops)

Why 2n states?

→ 0's are shifted in until all flipflops become 1

All inverted.

→ ~~1's are~~ shifted in until all FF become 0

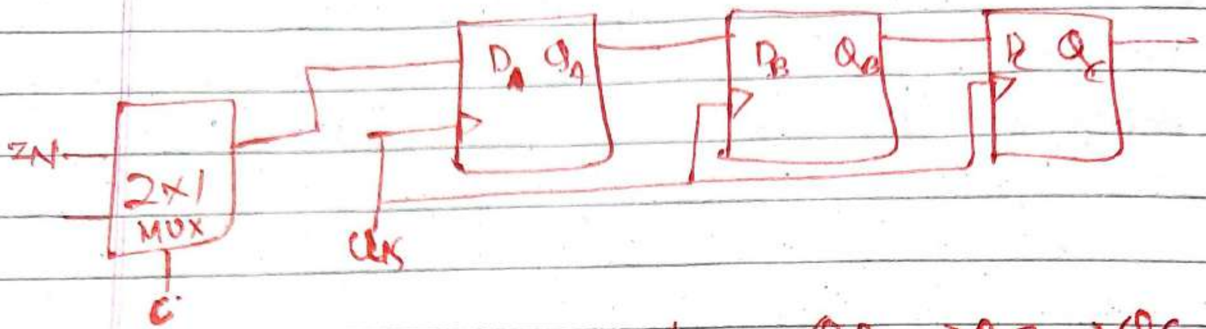
Filling with 1's → n transition

Filling with 0's → n transition

$\overline{2n}$

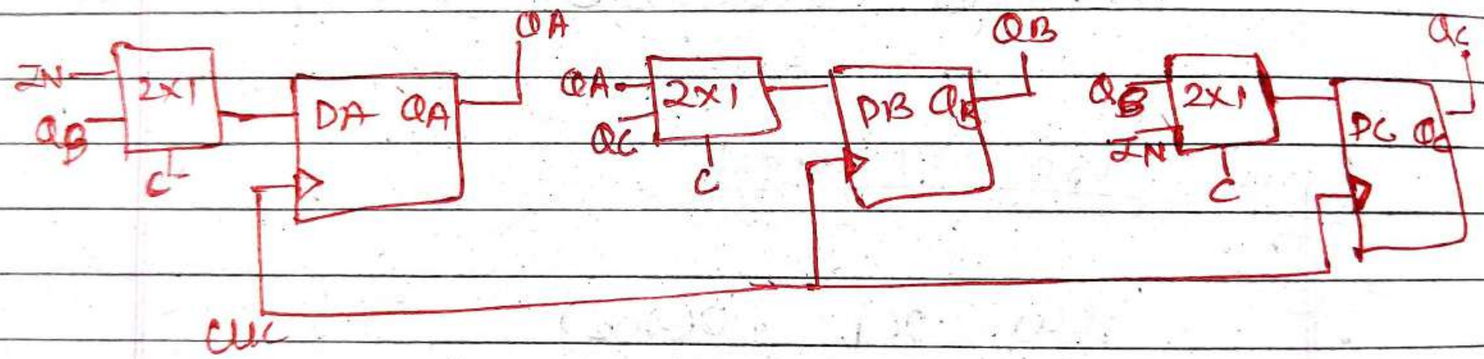
→ unused states = $\underbrace{2^n}_{\text{Total}} - \underbrace{2n}_{\text{used}}$

Q 3bit shift Reg. using 2x1 MUX & D flip flop



(C=0) right shift $QA \rightarrow QB \rightarrow QC$

(C=1) Left shift $QC \rightarrow QB \rightarrow QA$



Q 3bit Gray Counter

1

$b \rightarrow G$

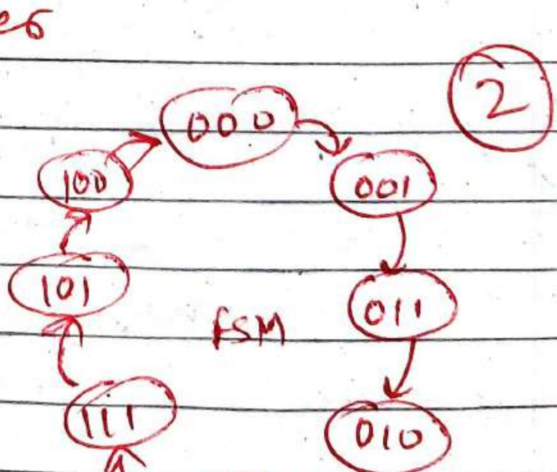
gray = ~~bin ⊕ bin~~

010 011

011 010

b	G
000	000
001	001
010	011
011	010
100	110
101	111
110	101
111	100

2



$\overline{AB} = \overline{A} + \overline{B}$

Excitation table of T F.F $= Q_n \oplus Q_{n+1}$

(3)

PS (MSB … LSB)			N.S			inputs		
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	D_2	D_1	D_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	1	0	1	1
0	1	0	0	1	0	0	1	0
0	1	1	1	1	0	1	1	0
1	0	0	1	1	1	1	1	1
1	0	1	1	0	1	1	0	1
1	1	0	1	0	0	1	0	0
1	1	1	0	0	0	0	0	0

D_2

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0			1	
1	1	1		1

$D_1 =$

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0		1	1	1
1	1			

$D_0 =$

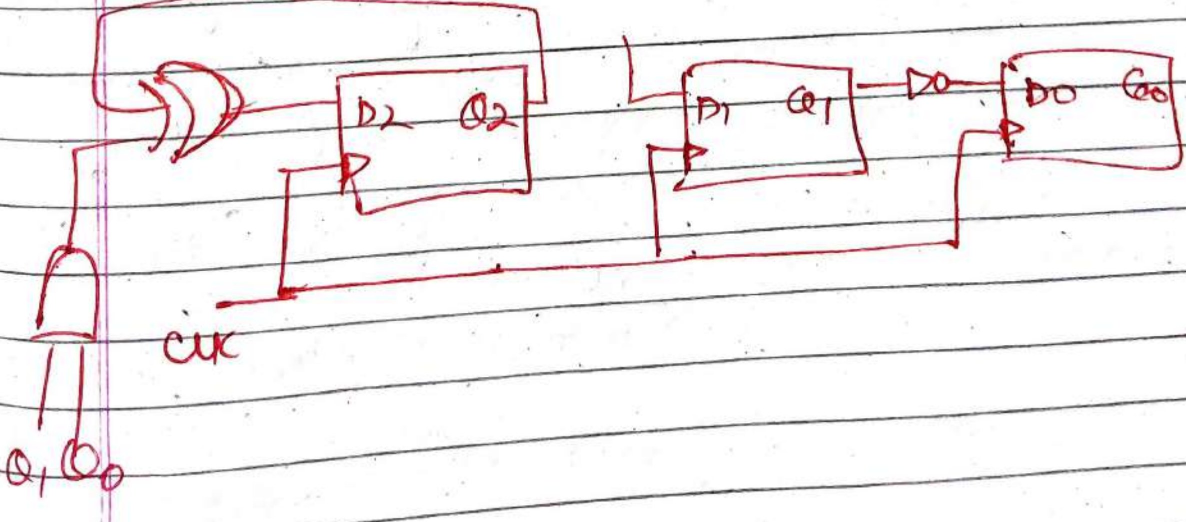
	00	01	11	10
0	1	1		
1	1	1		

$= \overline{Q_1}$

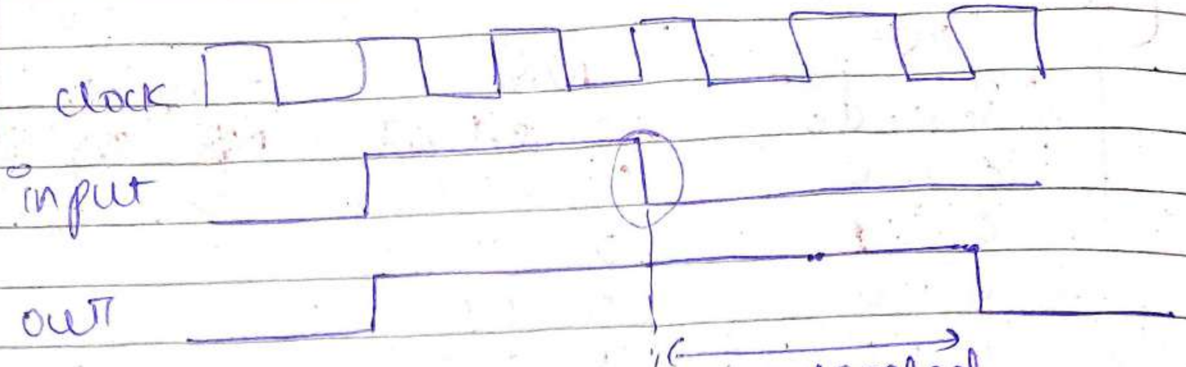
$Q_2\overline{Q_0} + Q_2\overline{Q_1} + \overline{Q_2}(Q_1Q_0)$ $\quad Q_2\overline{Q_0} + \overline{Q_2}Q_1$

$Q_2(\overline{Q_1} + \overline{Q_0}) + \overline{Q_2}(Q_1Q_0)$

$Q_2(\overline{Q_1Q_0}) + \overline{Q_2}(Q_1Q_0)$



Q



extended for 2 clock cycles.

circuit?

Ask yourself

↳ can combinational logic count ~~coun~~ clock cycles?

∴ NO

only sequential circuit (FF, counter, FSM, Registers)

Requirement → delay only falling edge of input

Hardware needed: falling edge detector, 2 bit counter

00
01
10
11

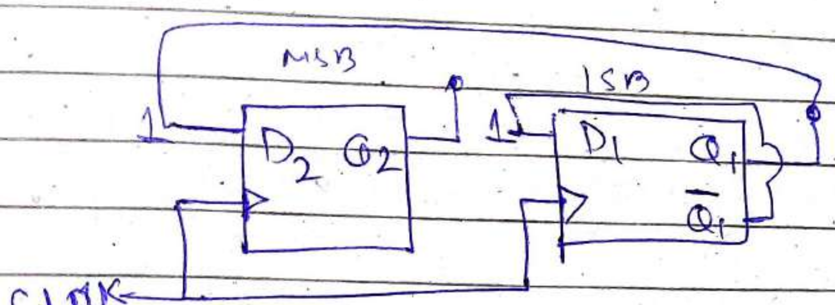
① Counter based design

Q_2Q_1	Preset
0 0	0
0 1	0
1 0	1
1 1	X

$Q_2 \backslash Q_1$	0	1
0		
1	1	X

$PRESET = Q_2$

PS Q_2Q_1	NS Q_2Q_1	D_2D_1
00	01	0 1
01	10	1 0
10	X	X X
11	X	X X



$D_2 =$

$Q_2 \backslash Q_1$	0	1
0		1
1	X	X

$= Q_1$

$D_1 =$

$Q_2 \backslash Q_1$	0	1
0	1	
1	X	X

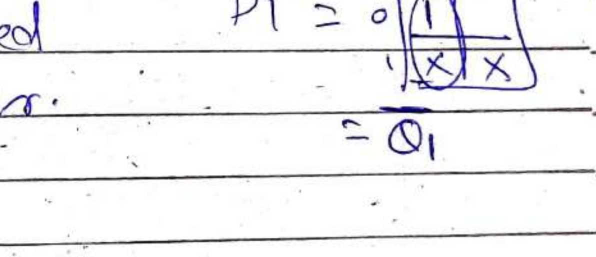
$= \overline{Q_1}$

② Without counter Based via Shift Register.

→ First falledge detection

prev.	current	OUT
0	0	0
0	1	0
1	0	1
1	1	0

fall Edge = prev · ($\overline{\text{current}}$)

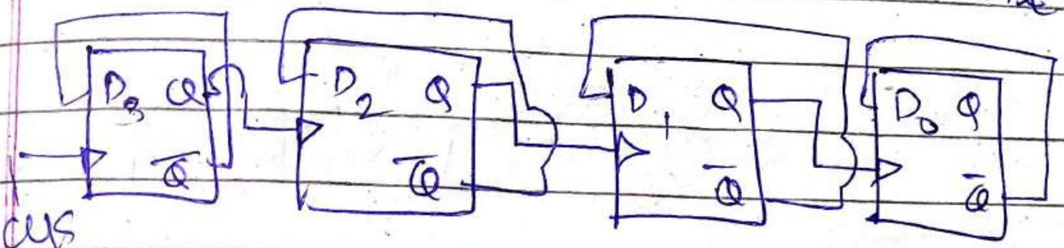


→ Now delay by 2 clock
Pass the pulse through 2 (D flip flops)

Q Design BCD counter 0 to 9999 using BCD decade counter

count 0 to 9 = 10 $\leq 2^{4}$ → 4 flip flops require

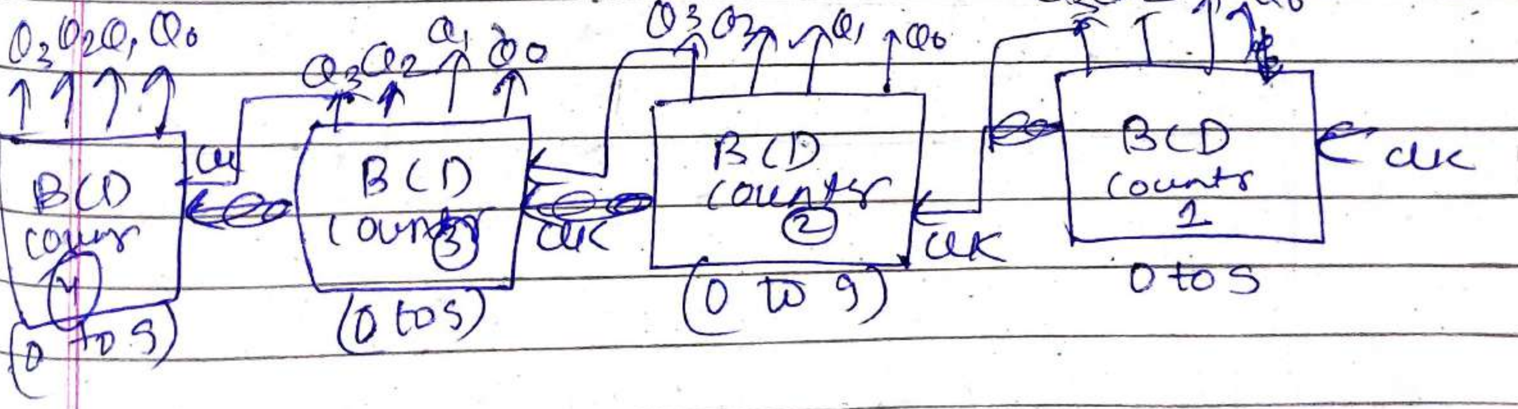
∴ 4 bit



0 to 9999 → $\underline{1000} \leq 2^{n}$ → 10 flip flops

if No BCD decade counter

But using BCD decade counter.

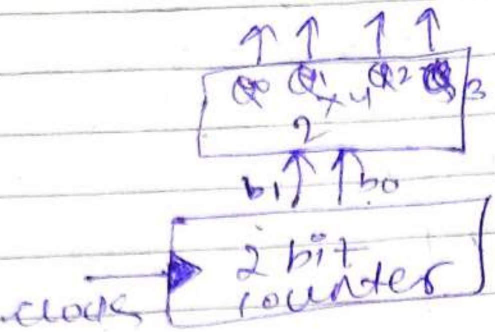


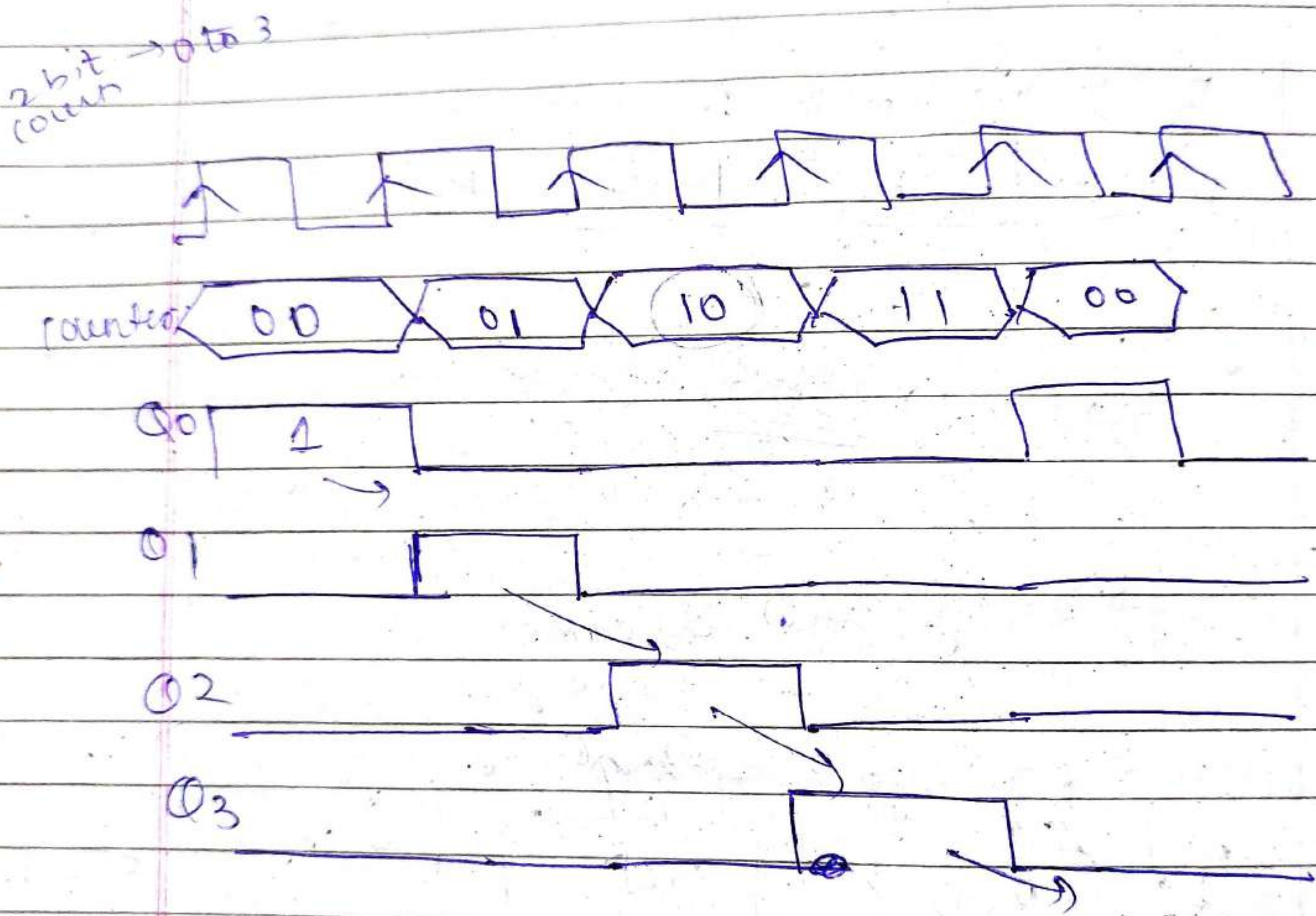
cascading of all BCD decade counter

~~0110 = 10000~~

MOD $10 \times 10 \times 10 \times 10$ = MOD 1000

Q Ring Counter implemented as

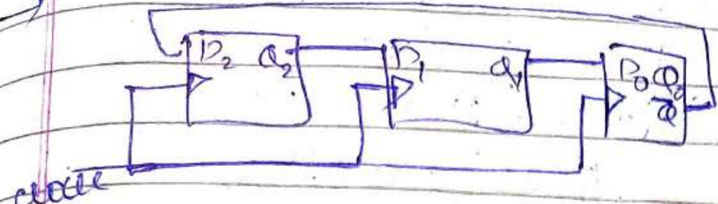




To generate 8 timing signals we need 3 bit Counter & 3x8 Decoder

Q 3 bit Counter then unused states?

→



	Q_2	Q_1	Q_0
initial state	0	0	0
↑	1	1	1
↑	0	0	

All 3 flipflop will not receive updated value at a time

D_1 ko Q_2 ki old value reccieve hoyegi

	Q_2	Q_1	Q_0	
initial state	0	0	0	
↑	1	0	0	— 4
↑	1	1	0	— 6
↑	1	1	1	— 7
↑	0	1	1	— 3
↑	0	0	1	— 1
↑	0	0	0	— 0

length counting of sequence

2N

2×3

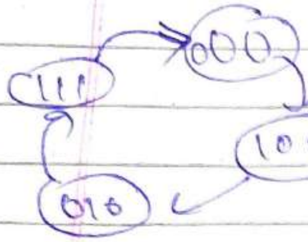
= 6

$2^3 = 8$

Total 0 to 7 value

But 2, 5 are unused state

Q Design synchronous Counter using D F.F in sequence 0, 4, 2, 7, 0, 4, 2, ...



P.S (MSB) Q_2	Q_1	Q_0	N.S Q_2^+	Q_1^+	Q_0^+	Input D_2	D_1	D_0
0	0	0	1	0	0	1	0	0
1	0	0	0	1	0	0	1	0
0	1	0	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0

$D_2 =$

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0	1			1
1				

$= \overline{Q_2}\,\overline{Q_0}$

$D_1 =$

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0				1
1	1			

$= Q_2\overline{Q_1}\,\overline{Q_0} + \overline{Q_2}Q_1\overline{Q_0}$

$= \overline{Q_0}(Q_2\overline{Q_1} + \overline{Q_2}Q_1)$

$= \overline{Q_0}(Q_2 \oplus Q_1)$

$D_0 =$

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0				1
1				

$\overline{Q_2}\,Q_1\overline{Q_0}$